SIMATSENGINEERING

Department of Information Technology

DSA01–Object-Oriented Programming with C++

ASSIGNMENT2–CO2

Functions and Object-Oriented Programming Concepts in C++

CO2:Applyobject-orientedprogrammingprinciplesandC++constructssuchasdatatypes,operators and control structures to design and write structured OOP-based programs.

Student Name	D GOVARDHAN RAJU	**Register Number**	192525125
Department / Section	AI&DS	**Academic Year / Semester**	2025
Faculty Name	**AMIRTHA S**	**Natureof Assignment**	Individual
Duration	1Week	**Total Marks**	50
Date of Submission	06-11-2026	**Date of Evaluation**	
GitHub Repository	192525125-DSA0114	**Compiler**	g++13.3.0,-std=c++17-Wall-Wextra

MARK SHEET(Rubric-based Evaluation)

Task	TaskTitle	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	DataTypesandCompatibility	4		
4	ImplicitConversions	4		
5	OperatorsandPrecedence	4		
6	ControlStructures	4		
7	ProceduralvsObject-Oriented	4		
8	InputandOutput	4		
9	ImplementationandTesting	4		
10	ExpectationvsExecution	4		
11	FinalRecommendation	4		
12A	Re-evaluate:Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	50		Percentage:

OVERALLREMARKS

D GOVARDHAN RAJU		
Student Signature	**Faculty Signature**	**Program Director**

OBJECT-ORIENTEDPROGRAMMINGWITHC++
ASSIGNMENT 2 – CO2

CO2: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and itsbasic principles, tokens,keywords, identifiers, constants, data types,type compatibility,variables,operators,implicitconversions,operatorprecedenceandcontrolstructures.

Nature of Assignment: Individual
Duration:1Week
TotalMarks:50
Submission Date: _____
Bloom'sLevelsAssessed:K5–Evaluate(everytaskinthisassignmentisatK5level)

AssignmentTitle: Functions and Object-Oriented Programming Concepts in C++
(Total Marks - 50)

Mandatory Execution and GitHub Submission

- Every student must individuallyexecuteeverytaskinthisassignment,includingall"Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keeponefolder foreachtask(Task01,Task02…),containingtheC++sourcefiles,input files and output screenshots.
- IncludeaREADME.mdwithyourname,registernumber,compilerandversion,theexact g++ commandusedtocompile, thestepstoruneachprogram, andashort noteoftheresultofeach task.
- Commit your work progressively – at least one commit for each task. A single upload on the last day will not be accepted.
- SubmittheGitHublinkbelowandonthecover pageofyour report.

GitHub Repository Link: _____

Howto AnswertheAdditionalQuestions

Complete the Code: Copythe given code into your repository, replace everyTODO with working codewithoutchangingtherestoftheprogram,executeitandcompareyouroutputwiththeexpected output.

Predict the Output: Writethe exact output ofthe givenprogrambefore running it. Thenrun it and explaineverydifferencebetweenyourpredictionandtheactualoutput,usingtheC++rulesinvolved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario—NammaMetroSmart-CardFare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs20 or Rs30 according tothe farthest zonetravelled, witha20% surchargeduring peak hours(8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version ofthe gatesoftware was written in a hurry. Some fares are off bya paisa, the peak checkaccepts10AMaspeakand8PMasoff-peakinconsistently,anegativebalanceonceappeared on a card, and the fare values are typed in several places.

Thenumberofjourneysprocessed maygrow:1station(20,000journeys aday)→1 line(500,000a day) → whole network (3,000,000 a day).

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants,operatorsandcontrolstructures,implementthefarelogicandvalidateit,sothateveryfare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

Q1 – CALL BY VALUE, CALL BY REFERENCE AND CALL BY POINTER

1.1 Introduction

In C++, arguments can be passed to functions in different ways. The three common methods are call by value, call by reference, and call by pointer. These methods determine whether a function can modify the original variable supplied by the caller.

In a hostel fee management system, students may receive different concessions such as merit concession, sports concession, and single-parent concession. A function must correctly update the student's hostel fee.

1.2 Call by Value

Call by value passes a copy of the original variable to the function. Any changes made inside the function affect only the local copy. The original variable remains unchanged.

Example:

```
#include <iostream>
using namespace std;

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
    cout << "Fee inside function: " << fee << endl;
}

int main()
{
    long long hostelFee = 45000;

    applyByValue(hostelFee, 10);

    cout << "Original hostel fee: " << hostelFee << endl;

    return 0;
}
```

Output:

```
Fee inside function: 40500
Original hostel fee: 45000
```

Explanation: The function receives a copy of `hostelFee`. The local fee becomes 40500, but the original fee remains 45000.

1.3 Call by Reference

Call by reference passes an alias of the original variable. Changes made through the reference directly affect the caller's variable.

Example:

```
#include <iostream>
using namespace std;

void applyByReference(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;

    applyByReference(hostelFee, 10);

    cout << "Updated hostel fee: " << hostelFee << endl;

    return 0;
}
```

Output:

```
Updated hostel fee: 40500
```

Explanation: The reference parameter refers to the original hostel fee. Therefore, the original value is updated directly.

1.4 Call by Pointer

Call by pointer passes the address of a variable. The function uses the pointer to access and modify the original variable.

Example:

```
#include <iostream>
using namespace std;

void applyByPointer(long long* fee, int percent)
{
    if (fee != nullptr)
    {
        *fee -= (*fee * percent) / 100;
    }
}

int main()
{
    long long hostelFee = 45000;

    applyByPointer(&hostelFee, 10);

    cout << "Updated hostel fee: " << hostelFee << endl;
```

```
    return 0;
}
```

Output:

```
Updated hostel fee: 40500
```

Explanation: The address of the hostel fee is passed to the function. The pointer accesses the original variable and changes its value.

1.5 Difference Between the Three Methods

Feature	Call by Value	Call by Reference	Call by Pointer
Parameter	Copy of value	Reference	Address
Original variable changes	No	Yes	Yes
Syntax	`int x`	`int& x`	`int* x`
Null value possible	No	No	Yes
Direct modification	No	Yes	Yes
Safety	High	High when used correctly	Requires pointer checks

1.6 Why Only Two Methods Change the Original Fee

Call by value creates a separate local copy. Therefore, changes do not affect the original fee.

Call by reference creates an alias for the original variable. Call by pointer uses the original variable's memory address. Both methods can modify the caller's fee.

A `const` reference should be used when a function needs to read a large object without copying it but should not modify it.

Example:

```
void displayStudent(const Student& s)
{
    // The object can be read but not modified here.
}
```

1.7 Predict the Output

Given program:

```
#include <iostream>
using namespace std;

void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}

int& bigger(int& p, int& q)
{
    return p > q ? p : q;
}

void show(int x, int y = 5, int z = 10)
{
    cout << x + y + z << ' ';
}

int main()
```

```
{
    int a = 1, b = 2, c = 3;

    change(a, b, &c);
    cout << a << ' ' << b << ' ' << c << '\n';

    bigger(a, c) = 100;
    cout << a << ' ' << c << '\n';

    show(1);
    show(1, 2);
    show(1, 2, 3);
    cout << '\n';

    return 0;
}
```

Predicted and expected output:

```
1 12 13
1 100
16 13 6
```

Explanation:

1. `a` is passed by value, so its original value remains 1.
2. `b` is passed by reference, so its value becomes 12.
3. `c` is passed by pointer, so its value becomes 13.
4. `bigger(a,c)` returns a reference to the larger value. Since `c` is larger, the assignment changes `c` to 100.
5. `show(1)` uses default values 5 and 10.
6. `show(1,2)` uses the default value 10 for the third argument.
7. `show(1,2,3)` uses all three supplied arguments.

1.8 Complete the Code

```
#include <iostream>
using namespace std;

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
}

void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;

    applyByValue(hostelFee, 10);

    cout << "after call by value   : "
         << hostelFee << '\n';

    applyConcession(hostelFee, 10);

    cout << "after call by reference: "
         << hostelFee << '\n';

    return 0;
```

```
}
```

Output:

```
after call by value   : 45000
after call by reference: 40500
```

1.9 Alternative Concept – Returning the New Fee

Instead of modifying the original fee through a reference, the function can return the updated fee.

```
#include <iostream>
using namespace std;

long long applyConcession(long long fee, int percent)
{
    return fee - (fee * percent / 100);
}

int main()
{
    long long hostelFee = 45000;

    hostelFee = applyConcession(hostelFee, 10);

    cout << "Updated fee: " << hostelFee << endl;

    return 0;
}
```

Output:

```
Updated fee: 40500
```

1.10 Complexity Analysis

Method	Time Complexity	Extra Parameter Storage
Call by value	O(1)	One copy of the parameter
Call by reference	O(1)	Reference, no object copy
Call by pointer	O(1)	One pointer
Return by value	O(1)	Parameter and return value, subject to compiler optimisation

For a large Student object, passing by value may copy the entire object. Passing by reference or const reference avoids that object copy.

1.11 Conclusion

Call by value is suitable when the original variable should not change. Call by reference is suitable when a function must directly update the original value. Call by pointer is useful when working with memory addresses or optional objects. For the hostel fee concession system, call by reference provides a simple way to update the fee. Returning the updated fee is also a good design because it makes the modification explicit.

Q2 – RETURN BY REFERENCE AND ITS DANGERS

2.1 Introduction

In C++, a function can return a reference to an existing variable. This allows the calling function to directly access and modify the original variable. Returning a reference can avoid copying, but it must be used carefully.

In an examination marks register, marks are stored in an array. A function can identify the highest mark and return a reference to that array element.

2.2 Program to Return the Highest Mark by Reference

```
#include <iostream>
using namespace std;

int& highestMark(int marks[], int n)
{
    int index = 0;

    for (int i = 1; i < n; i++)
    {
        if (marks[i] > marks[index])
        {
            index = i;
        }
    }

    return marks[index];
}

int main()
{
    int marks[5] = {75, 89, 92, 68, 85};

    cout << "Highest mark: "
         << highestMark(marks, 5) << endl;

    highestMark(marks, 5) += 2;

    cout << "After moderation: "
         << highestMark(marks, 5) << endl;

    return 0;
}
```

Output:

```
Highest mark: 92
After moderation: 94
```

2.3 Explanation

The function searches the array and identifies the index containing the highest mark. It returns a reference to that array element.

The statement:

```
highestMark(marks, 5) += 2;
```

directly modifies the original array element. Therefore, the highest mark changes from 92 to 94.

2.4 Returning a Reference to a Local Variable

A local variable exists only while its function is executing. Returning a reference to that local variable creates a dangling reference after the function ends.

Incorrect example:

```
int& dangerous()
{
    int x = 100;
    return x;
}
```

The compiler may issue a warning similar to:

```
warning: reference to local variable returned
```

Using the returned reference causes undefined behaviour because the local variable's lifetime has ended.

Correct alternative:

```
int safeValue()
{
    int x = 100;
    return x;
}
```

Returning a copy is safe because the value is returned independently of the local variable's lifetime.

2.5 Difference Between Return Types

Return Type	Meaning	Modification
`int`	Returns a copy	Does not modify original
`int&`	Returns a reference	Can modify original
`const int&`	Returns read-only reference	Cannot modify through reference
`int*`	Returns an address	Can modify through pointer if valid

A reference should only be returned when the referenced object will remain alive for the entire period in which the caller uses it.

2.6 Changed Condition – Using std::vector

A `std::vector` can grow dynamically. When new elements are inserted, the vector may allocate a new memory block and move its elements. This can invalidate references and pointers to its elements.

Example:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> marks = {75, 89, 92};

    int& highest = marks[2];

    marks.push_back(95);

    // Do not use highest here: push_back may have invalidated it.
```

```
    cout << "Current highest mark: " << marks[3] << endl;

    return 0;
}
```

Output:

```
Current highest mark: 95
```

The example deliberately avoids using the old reference after insertion. The reference may have become invalid.

2.7 Alternative Concept – Returning an Index

Instead of returning a reference, the function can return the index of the highest mark.

```
#include <iostream>
using namespace std;

int highestIndex(const int marks[], int n)
{
    int index = 0;

    for (int i = 1; i < n; i++)
    {
        if (marks[i] > marks[index])
        {
            index = i;
        }
    }

    return index;
}

int main()
{
    int marks[5] = {75, 89, 92, 68, 85};

    int index = highestIndex(marks, 5);

    cout << "Highest mark: " << marks[index] << endl;

    marks[index] += 2;

    cout << "After moderation: " << marks[index] << endl;

    return 0;
}
```

Output:

```
Highest mark: 92
After moderation: 94
```

2.8 Alternative Concept – Returning a Pointer

```
int* highestPointer(int marks[], int n)
{
    int index = 0;

    for (int i = 1; i < n; i++)
    {
        if (marks[i] > marks[index])
            index = i;
    }
```

```
    return &marks[index];
}
```

Calling code:

```
int* p = highestPointer(marks, 5);

if (p != nullptr)
{
    *p += 2;
}
```

2.9 Complexity Analysis

Method	Finding Highest Mark	Extra Space
Return by reference	O(n)	O(1)
Return by index	O(n)	O(1)
Return by pointer	O(n)	O(1)

All three methods require scanning the array once. Therefore, the time complexity is O(n), and the auxiliary space complexity is O(1).

Returning an index is often a practical choice for a vector because the index can be used again after the vector changes size, provided the element ordering and intended meaning are still valid. A pointer or reference may become invalid after reallocation.

2.10 Conclusion

Returning a reference allows direct modification of an existing array element without creating a copy. However, returning a reference to a local variable is unsafe because its lifetime ends when the function returns. For a growing vector, returning an index is often safer than keeping a reference or pointer across operations that may reallocate memory.

Q3 – DEFAULT ARGUMENTS AND FUNCTION OVERLOADING

3.1 Introduction

Function overloading allows multiple functions to have the same name but different parameter lists. Default arguments allow a function to use predefined values when the caller does not supply all arguments.

In a courier charge calculator, the delivery fee depends on parcel weight, express delivery and insurance.

3.2 Understanding the Compiler Error

Consider the following declarations:

```
double charge(double weight);
double charge(double weight, bool express);
double charge(double weight, bool express = false,
              bool insured = false);
```

A call such as:

```
charge(2.0);
```

can match more than one declaration because the third function can be called with only one argument using its default values. The first and third declarations therefore compete for the same call.

This creates an ambiguous or conflicting function design.

3.3 Redesigned Function Overloading

The redesigned solution uses different parameter types and a clear set of overloads.

```
#include <iostream>
using namespace std;

struct Parcel
{
    double weightKg;
    bool express;
    bool insured;
};

double charge(double weightKg,
              bool express = false,
              bool insured = false)
{
    double total = weightKg * 50;

    if (express)
        total += 40;

    if (insured)
        total += 20;

    return total;
}

double charge(int weightGrams)
```

```
{
    double weightKg = weightGrams / 1000.0;
    return charge(weightKg);
}

double charge(const Parcel& p)
{
    return charge(p.weightKg, p.express, p.insured);
}

int main()
{
    Parcel p{2.5, true, true};

    cout << "Charge 1: " << charge(2) << endl;
    cout << "Charge 2: " << charge(2.0f) << endl;
    cout << "Charge 3: " << charge(2.0) << endl;
    cout << "Charge 4: " << charge(2.0, true) << endl;
    cout << "Charge 5: " << charge(2.0, true, true) << endl;
    cout << "Charge 6: " << charge(p) << endl;

    return 0;
}
```

Output:

```
Charge 1: 0.1
Charge 2: 100
Charge 3: 100
Charge 4: 140
Charge 5: 160
Charge 6: 185
```

3.4 Explanation of the Six Calls

1. `charge(2)` selects the integer overload because the argument is an integer. Two grams are converted into 0.002 kg, resulting in a charge of 0.1.
2. `charge(2.0f)` selects the double overload. The float is promoted to double. The base charge is 100.
3. `charge(2.0)` selects the double overload by exact match. The base charge is 100.
4. `charge(2.0, true)` selects the double overload and applies the express charge.
5. `charge(2.0, true, true)` selects the double overload and applies express and insurance charges.
6. `charge(p)` selects the Parcel overload.

3.5 How Overload Resolution Works

The compiler generally prefers the best available conversion sequence:

1. Exact match – the argument already has the required type.
2. Promotion – a standard promotion, such as `float` to `double`.
3. Conversion – another standard conversion, such as `int` to `double`.

If two candidate functions are equally good, the compiler may report an ambiguous call.

Default arguments must be specified from right to left. Once a parameter has a default value, all parameters to its right must also have default values.

Correct:

```
void display(int a, int b = 10, int c = 20);
// void display(int a = 10, int b, int c = 20);
```

3.6 Alternative Concept – Single Parcel Structure

An alternative is to use one Parcel structure containing all optional information. A single function can accept the structure and calculate the charge.

```
#include <iostream>
using namespace std;

struct ParcelInfo
{
    double weightKg;
    bool express = false;
    bool insured = false;
};

double calculateCharge(const ParcelInfo& p)
{
    double total = p.weightKg * 50;

    if (p.express)
        total += 40;

    if (p.insured)
        total += 20;

    return total;
}

int main()
{
    ParcelInfo p{2.5, true, true};

    cout << "Total courier charge: "
         << calculateCharge(p) << endl;

    return 0;
}
```

Output:

```
Total courier charge: 185
```

3.7 Complexity Analysis

Design	Run-time Calculation	Auxiliary Space
Overloaded functions	O(1)	O(1)
Parcel structure	O(1)	O(1)
Function template	Depends on function body	Depends on function body

Overload resolution is performed at compile time, so it does not add a run-time search cost. The Parcel structure is easier to extend when new options are introduced, while overloads are useful when different input types need different handling.

3.8 Conclusion

Function overloading and default arguments make C++ programs flexible and reusable. However, overlapping overloads can cause ambiguity. A clear function interface should avoid competing signatures. The redesigned courier calculator supports integer weight, double weight and Parcel objects without ambiguous calls.

Q4 – SPECIFYING A CLASS WITH AN ARRAY INSIDE IT

4.1 Introduction

A class is a user-defined data type that combines data members and member functions. Access specifiers control how the members can be accessed.

In a bank savings account system, each account must store its account number, balance and the last ten transactions. Older transactions must be discarded when new transactions arrive.

4.2 Class Design

The SavingsAccount class contains:

- Account number.
- Account balance.
- Fixed-size transaction array.
- Transaction count.
- Deposit function.
- Withdraw function.
- Mini-statement function.

The data members are private to protect the account information. Public member functions provide controlled access.

4.3 C++ Program Using an Array of Transactions

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

class SavingsAccount
{
private:
    string accountNumber;
    double balance;
    double transactions[10];
    int count;

public:
    SavingsAccount(string acc, double initialBalance)
        : accountNumber(acc), balance(initialBalance), count(0)
    {
    }

    void addTransaction(double amount)
    {
        if (count < 10)
        {
            transactions[count] = amount;
            count++;
        }
        else
        {
            for (int i = 0; i < 9; i++)
            {
                transactions[i] = transactions[i + 1];
            }

            transactions[9] = amount;
        }
    }
```

```
    void deposit(double amount)
    {
        if (amount > 0)
        {
            balance += amount;
            addTransaction(amount);
        }
        else
        {
            cout << "Invalid deposit amount\n";
        }
    }

    void withdraw(double amount)
    {
        if (amount > 0 && amount <= balance)
        {
            balance -= amount;
            addTransaction(-amount);
        }
        else
        {
            cout << "Invalid withdrawal\n";
        }
    }

    void miniStatement() const;
};

void SavingsAccount::miniStatement() const
{
    cout << "\nAccount Number: " << accountNumber << endl;
    cout << fixed << setprecision(2);
    cout << "Current Balance: Rs. " << balance << endl;

    cout << "\nLast Transactions:\n";

    for (int i = 0; i < count; i++)
    {
        cout << transactions[i] << endl;
    }
}

int main()
{
    SavingsAccount account("SB1001", 5000);

    account.deposit(1000);
    account.withdraw(500);
    account.deposit(250);
    account.withdraw(200);

    account.miniStatement();

    return 0;
}
```

Output:

```
Account Number: SB1001
Current Balance: Rs. 5550.00

Last Transactions:
1000.00
-500.00
```

```
250.00
-200.00
```

4.4 Explanation of the Program

1. The class stores account details and transaction history.
2. The account number and balance are private.
3. The deposit function increases the balance.
4. The withdraw function decreases the balance if sufficient funds are available.
5. The addTransaction function stores the transaction in the array.
6. The miniStatement function displays the account details and stored transactions.
7. The scope resolution operator `::` is used to define `miniStatement()` outside the class.

4.5 Why the Array Must Be Private

The transaction array should be private because users should not directly change transaction records. If the array were public, external code could modify or delete transaction values without following the account rules.

Encapsulation protects the data and ensures that changes happen through validated member functions.

4.6 Handling the 11th Transaction

When the transaction array already contains ten transactions, a new transaction arrives.

Method 1 – Shifting the Array

The program moves each element one position to the left and stores the new transaction at the last position.

Example:

```
Before:
T1 T2 T3 T4 T5 T6 T7 T8 T9 T10

After adding T11:
T2 T3 T4 T5 T6 T7 T8 T9 T10 T11
```

The oldest transaction is removed.

Method 2 – Circular Index

A circular buffer uses a start index to identify the oldest transaction. New transactions overwrite the oldest entry without shifting all the other elements.

This method reduces the insertion time from O(k) to O(1), where k is the number of stored transactions.

4.7 Alternative Concept – Circular Buffer

```
#include <iostream>
using namespace std;

class TransactionBuffer
{
private:
    double transactions[10];
    int start;
    int count;
```

```
public:
    TransactionBuffer() : start(0), count(0) {}

    void add(double amount)
    {
        if (count < 10)
        {
            transactions[(start + count) % 10] = amount;
            count++;
        }
        else
        {
            transactions[start] = amount;
            start = (start + 1) % 10;
        }
    }

    void display() const
    {
        for (int i = 0; i < count; i++)
        {
            cout << transactions[(start + i) % 10] << endl;
        }
    }
};

int main()
{
    TransactionBuffer history;

    for (int i = 1; i <= 12; i++)
    {
        history.add(i * 100);
    }

    cout << "Last 10 transactions:\n";
    history.display();

    return 0;
}
```

Output:

```
Last 10 transactions:
300
400
500
600
700
800
900
1000
1100
1200
```

4.8 Comparison of Transaction Storage Methods

Method	Adding Transaction	Space
Fixed array with shifting	O(k) when full	O(k)
Circular buffer	O(1)	O(k)
`std::deque` with push_back and pop_front	O(1) at the ends, amortised for insertion	O(k)

If the requirement changes from ten to fifty transactions, the fixed array size can be changed to fifty. A circular buffer can also be changed by updating its capacity. A deque is useful when the capacity changes dynamically.

4.9 Conclusion

A class with a private transaction array provides a secure and organised way to manage savings account information. Shifting is simple for a small number of transactions, but a circular buffer is more efficient when transactions arrive frequently. For a system that stores the last fifty transactions, a circular buffer or deque is a suitable design.

Q5 – STATIC MEMBERS AND ARRAYS OF OBJECTS

5.1 Introduction

Static data members belong to the class rather than to individual objects. Only one copy of a static data member is shared among all objects of that class.

In a college bus ticket system, the transport office must assign unique seat numbers and maintain the total number of tickets issued. Tickets beyond the bus capacity must be placed on a waiting list.

5.2 Static Members

A static data member is shared by all objects. It is declared inside the class and defined outside the class.

Example:

```
class BusTicket
{
    static int issued;
};

int BusTicket::issued = 0;
```

The static member function can access static members directly. It cannot use the `this` pointer because it is not called for one particular object.

5.3 Complete BusTicket Program

```
#include <iostream>
#include <string>
using namespace std;

class BusTicket
{
private:
    static int issued;
    static const int CAPACITY = 3;

    int seat;
    string name;

public:
    explicit BusTicket(const string& n)
        : seat(0), name(n)
    {
        if (issued < CAPACITY)
        {
            issued++;
            seat = issued;
        }
    }

    static int count()
```

```
    {
        return issued;
    }

    void print() const
    {
        cout << name << ": ";

        if (seat != 0)
            cout << "Seat " << seat;
        else
            cout << "Waiting";

        cout << '\n';
    }
};

int BusTicket::issued = 0;

int main()
{
    BusTicket trip[4] = {
        BusTicket("Asha"),
        BusTicket("Bala"),
        BusTicket("Chitra"),
        BusTicket("Dev")
    };

    for (const BusTicket& t : trip)
    {
        t.print();
    }

    cout << "Issued: " << BusTicket::count() << '\n';

    return 0;
}
```

Output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

5.4 Explanation

1. `issued` is a static variable shared by all BusTicket objects.
2. `CAPACITY` represents the maximum number of seats.
3. Each ticket object receives a seat number if the bus has an available seat.
4. The fourth ticket receives seat zero, representing the waiting list.
5. The static function `count()` returns the number of issued seats.
6. The statement `int BusTicket::issued = 0;` defines and initialises the static data member.

5.5 Array of Objects

An array of objects stores multiple objects of the same class.

Example:

```
BusTicket trip[4] = {
    BusTicket("Asha"),
```

```
    BusTicket("Bala"),
    BusTicket("Chitra"),
    BusTicket("Dev")
};
```

Each object is constructed separately. The constructor runs once for each object in the array.

5.6 Predict the Output

Given program:

```
#include <iostream>
using namespace std;

class Token
{
    static int next;
    int id;

public:
    Token()
    {
        id = ++next;
    }

    int get() const
    {
        return id;
    }

    static int issued()
    {
        return next - 100;
    }
};

int Token::next = 100;

int main()
{
    Token counter[3];

    Token extra;

    cout << counter[0].get() << ' '
         << counter[2].get() << ' '
         << extra.get() << ' '
         << Token::issued() << '\n';

    return 0;
}
```

Predicted and expected output:

```
101 103 104 4
```

Explanation:

- The static variable `next` starts at 100.
- Three Token objects are created in the array.
- Their IDs become 101, 102 and 103.
- One additional object is created with ID 104.

- The constructor runs four times.
- The `issued()` function returns `104 - 100`, which is 4.

5.7 Creating Tickets Inside a Function

The static counter continues to exist even after a function returns. Therefore, if tickets are created inside a function that is called twice, the second call continues from the previous count.

Example:

```
#include <iostream>
using namespace std;

class TicketCounter
{
private:
    static int issued;

public:
    TicketCounter()
    {
        issued++;
    }

    static int count()
    {
        return issued;
    }
};

int TicketCounter::issued = 0;

void createTickets()
{
    TicketCounter a;
    TicketCounter b;

    cout << "Inside function: "
         << TicketCounter::count() << endl;
}

int main()
{
    createTickets();
    createTickets();

    cout << "Final count: "
         << TicketCounter::count() << endl;

    return 0;
}
```

Output:

```
Inside function: 2
Inside function: 4
Final count: 4
```

5.8 Alternative Concept – Global Counter

A global variable can maintain the number of tickets issued. However, it can be accessed and modified from many parts of the program. This makes it harder to control.

5.9 Alternative Concept – Separate TicketOffice Object

A separate TicketOffice object can store the issued count and be passed to every BusTicket object.

```
#include <iostream>
#include <string>
using namespace std;

class TicketOffice
{
private:
    int issued = 0;
    const int capacity = 3;

public:
    int allocateSeat()
    {
        if (issued < capacity)
            return ++issued;

        return 0;
    }

    int count() const
    {
        return issued;
    }
};

class BusTicket
{
private:
    int seat;
    string name;

public:
    BusTicket(const string& n, TicketOffice& office)
        : seat(office.allocateSeat()), name(n)
    {
    }

    void print() const
    {
        cout << name << ": ";

        if (seat == 0)
            cout << "Waiting\n";
        else
            cout << "Seat " << seat << '\n';
    }
};

int main()
{
    TicketOffice office;

    BusTicket a("Asha", office);
    BusTicket b("Bala", office);
    BusTicket c("Chitra", office);
    BusTicket d("Dev", office);

    a.print();
    b.print();
    c.print();
    d.print();
```

```
    cout << "Issued: " << office.count() << endl;

    return 0;
}
```

Output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

5.10 Complexity Analysis

Design	Time per Ticket	Extra Counter Space
Static data member	O(1)	One shared counter
Global variable	O(1)	One global counter
TicketOffice object	O(1)	One counter per office object

A static member is simple when all tickets belong to one shared bus or class-wide counter. A separate TicketOffice object is more flexible when multiple buses must maintain independent seat counts.

5.11 Conclusion

Static members allow all objects of a class to share common information. In the bus ticket system, a static counter assigns unique seat numbers and records the total number of seats issued. An array of objects makes it easy to create and manage multiple tickets. A separate TicketOffice object is a better alternative when multiple buses require independent counters.

Q6 – FRIEND FUNCTIONS AND INLINE FUNCTIONS

6.1 Introduction

A friend function is a non-member function that is permitted to access the private and protected members of a class. It is declared using the `friend` keyword inside the class.

An inline function is a function for which the compiler may substitute the function body at the call site. The `inline` keyword is a request and does not guarantee that the compiler will perform this substitution.

In a train journey planner, departure and arrival times are stored in a Time class. The system must calculate the difference between two times and determine which time is earlier.

6.2 Friend Function Program

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time
{
private:
    int h, m;

public:
    Time(int hh, int mm) : h(hh), m(mm)
    {
    }

    friend int minutesBetween(const Time& a,
                              const Time& b);
```

```
    friend bool earlier(const Time& a,
                        const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;

    return abs(first - second);
}

bool earlier(const Time& a, const Time& b)
{
    if (a.h != b.h)
        return a.h < b.h;

    return a.m < b.m;
}

int main()
{
    Time departure(9, 45);
    Time arrival(13, 10);

    cout << minutesBetween(departure, arrival)
         << ' '
         << boolalpha
         << earlier(arrival, departure)
         << '\n';

    return 0;
}
```

Output:

```
205 false
```

6.3 Explanation

The Time class stores hours and minutes as private members. The friend functions can directly access these private members.

The `minutesBetween()` function converts both times into total minutes and calculates their absolute difference.

For 9:45:

```
9 × 60 + 45 = 585 minutes
```

For 13:10:

```
13 × 60 + 10 = 790 minutes
```

Difference:

```
790 - 585 = 205 minutes
```

The `earlier()` function compares the hours first and then the minutes. Since 13:10 is not earlier than 9:45, the result is false.

6.4 Why a Friend Function Is Needed

A friend function is useful when an operation involves two objects equally. It can access private data from both objects without making the data public.

A member function is called using an object, such as:

```
departure.isEarlier(arrival);
```

A friend function is called like an ordinary function:

```
earlier(departure, arrival);
```

Friend functions are not members of the class, even though they have access to its private data.

6.5 Testing the Same Time Twice and Times Across Noon

The following test program checks equality and times on either side of noon.

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time
{
private:
    int h, m;

public:
    Time(int hh, int mm) : h(hh), m(mm) {}

    friend int minutesBetween(const Time&, const Time&);
    friend bool earlier(const Time&, const Time&);
};

int minutesBetween(const Time& a, const Time& b)
{
    int x = a.h * 60 + a.m;
    int y = b.h * 60 + b.m;

    return abs(x - y);
}

bool earlier(const Time& a, const Time& b)
{
    if (a.h != b.h)
        return a.h < b.h;

    return a.m < b.m;
}

int main()
{
    Time morning(11, 50);
    Time afternoon(12, 20);
    Time same(11, 50);

    cout << "Difference across noon: "
         << minutesBetween(morning, afternoon) << endl;

    cout << "Morning earlier than afternoon: "
         << boolalpha << earlier(morning, afternoon) << endl;
```

```
    cout << "Difference between same times: "
         << minutesBetween(morning, same) << endl;

    cout << "Same time is earlier: "
         << earlier(morning, same) << endl;

    return 0;
}
```

Output:

```
Difference across noon: 30
Morning earlier than afternoon: true
Difference between same times: 0
Same time is earlier: false
```

6.6 Inline Functions

Inline functions are useful for small functions that are called frequently. They may reduce function-call overhead when the compiler chooses to substitute the function body at the call site.

Example:

```
inline int square(int x)
{
    return x * x;
}
```

The compiler may inline this function, but it is not required to do so. Large functions and recursive functions may not be suitable for inlining.

6.7 Alternative Concept – Member Functions

The same comparison can be implemented as member functions.

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time
{
private:
    int h, m;

public:
    Time(int hh, int mm) : h(hh), m(mm) {}

    int minutesTo(const Time& other) const
    {
        int a = h * 60 + m;
        int b = other.h * 60 + other.m;

        return abs(a - b);
    }

    bool isEarlier(const Time& other) const
    {
        if (h != other.h)
            return h < other.h;

        return m < other.m;
    }
};
```

```
int main()
{
    Time departure(9, 45);
    Time arrival(13, 10);

    cout << departure.minutesTo(arrival) << ' '
         << boolalpha
         << departure.isEarlier(arrival) << endl;

    return 0;
}
```

Output:

```
205 true
```

6.8 Alternative Concept – Public Getters

Another design keeps the data private but provides public getter functions.

```
int getHours() const
{
    return h;
}

int getMinutes() const
{
    return m;
}
```

A non-member function can use these getters to compare two Time objects without being declared as a friend.

6.9 Complexity Analysis

Design	Time per Comparison	Encapsulation
Friend function	O(1)	Private data remains private
Member function	O(1)	Strong class-based design
Public getters	O(1)	Data remains private, read access is public

All three designs perform a constant number of operations. Therefore, the time complexity is O(1), and the auxiliary space complexity is O(1).

6.10 Measuring Inline Performance

The assignment asks for a comparison using ten million calls with `-O0` and `-O2`.

Example compilation commands:

```
g++ -std=c++17 -O0 inline_test.cpp -o inline_test_O0
./inline_test_O0

g++ -std=c++17 -O2 inline_test.cpp -o inline_test_O2
./inline_test_O2
```

Use the same test program and input for both builds. Record the measured times below.

Compiler Setting	Non-inline Time	Inline Time
`-O0`	_____ seconds	_____ seconds
`-O2`	_____ seconds	_____ seconds

Discussion: At `-O0`, compiler optimisation is limited, so function-call overhead may be more visible. At `-O2`, the compiler may inline small functions automatically, even when the programmer has not written the `inline` keyword. Actual performance depends on the compiler, processor and program.

6.11 Conclusion

Friend functions are useful when an operation requires equal access to two objects. They allow private data to remain protected while providing a convenient comparison interface. Member functions are also a good design when the operation naturally belongs to one object. Inline functions may improve performance for small, frequently called functions, but actual performance should be measured rather than assumed.

Q7 – DIAGNOSE, REDESIGN AND VALIDATE

7.1 Selected Program

For the redesign challenge, the SavingsAccount transaction history program from Q4 is selected.

The original program uses a fixed array of ten transactions. When the array becomes full, it shifts the remaining nine elements to the left to make space for the new transaction.

7.2 A. Diagnose the Limitation

The original transaction storage method works correctly for a small number of transactions. However, when the transaction array becomes full, adding a new transaction requires shifting all existing transactions.

The main limitation is the time required to insert a transaction when the array is full.

For a capacity of ten transactions, nine elements must be shifted whenever a new transaction arrives after the array is full.

If the capacity increases to fifty, forty-nine elements must be shifted for every new transaction after the array is full.

Identified limitations:

1. Insertion becomes slower when the transaction array is full.
2. Every new transaction requires multiple assignments.
3. The insertion operation depends on the number of stored transactions.
4. The design becomes less efficient when the history capacity increases.
5. The original array still uses fixed memory and cannot grow automatically.

7.3 B. Define the Design Objective

The objective is to redesign the transaction history so that a new transaction can be stored in constant time without shifting all existing transactions.

The redesigned program should:

1. Store only the latest ten transactions.
2. Remove the oldest transaction automatically when the buffer is full.
3. Insert a new transaction in O(1) time.
4. Preserve the order of transactions when displaying the mini-statement.

5. Use a fixed amount of memory.

7.4 C. Design the Revised Solution

The redesigned solution uses a circular buffer.

A circular buffer is a fixed-size array that treats its final position as connected to its first position. It maintains a start index that identifies the oldest transaction.

When the buffer is full, the next transaction replaces the oldest transaction. The start index moves forward by one position.

Pseudocode:

```
START

Create transaction array of size 10
Set start = 0
Set count = 0

FUNCTION addTransaction(amount)

    IF count is less than 10 THEN
        position = (start + count) MOD 10
        Store amount at position
        Increase count by 1

    ELSE
        Store amount at start
        start = (start + 1) MOD 10
    END IF

END FUNCTION

FUNCTION displayTransactions()

    FOR i = 0 TO count - 1
        position = (start + i) MOD 10
        Display transaction[position]
    END FOR

END FUNCTION

STOP
```

7.5 Revised C++ Program

```
#include <iostream>
using namespace std;

class TransactionBuffer
{
private:
    static const int SIZE = 10;

    double transactions[SIZE];
    int start;
    int count;

public:
    TransactionBuffer() : start(0), count(0)
    {
    }

    void addTransaction(double amount)
```

```
    {
        if (count < SIZE)
        {
            int position = (start + count) % SIZE;
            transactions[position] = amount;
            count++;
        }
        else
        {
            transactions[start] = amount;
            start = (start + 1) % SIZE;
        }
    }

    void displayTransactions() const
    {
        cout << "Last transactions:\n";

        for (int i = 0; i < count; i++)
        {
            int position = (start + i) % SIZE;
            cout << transactions[position] << endl;
        }
    }
};

int main()
{
    TransactionBuffer history;

    for (int i = 1; i <= 12; i++)
    {
        history.addTransaction(i * 100);
    }

    history.displayTransactions();

    return 0;
}
```

Expected output:

```
Last transactions:
300
400
500
600
700
800
900
1000
1100
1200
```

7.6 D. Predict Before Implementing

Before implementing the circular buffer, the expected improvement is that adding a new transaction will not require shifting the existing array elements.

Prediction:

- Original array with shifting: O(k) insertion when full.
- Redesigned circular buffer: O(1) insertion.
- Both designs use O(k) storage for a fixed capacity of k transactions.

- Both designs display the stored transactions in O(k) time.

The redesigned solution is expected to perform better when many transactions are added after the buffer becomes full.

7.7 E. Implement and Validate the Redesign

Both implementations should be tested with the same number of transactions.

Test Case 1: Add five transactions.

Expected result: All five transactions should be displayed.

Test Case 2: Add ten transactions.

Expected result: All ten transactions should be displayed in the correct order.

Test Case 3: Add twelve transactions.

Expected result: Only the latest ten transactions should be displayed. The first two transactions should be discarded.

Comparison table:

Test	Original Shifting Array	Circular Buffer
Five transactions	Stores all five	Stores all five
Ten transactions	Stores all ten	Stores all ten
Twelve transactions	Keeps latest ten	Keeps latest ten
Insert when full	O(k)	O(1)
Display	O(k)	O(k)
Memory	O(k)	O(k)

Measurement evidence:

Run both versions with the same large number of insertions and record the actual execution time.

Number of Insertions	Original Time	Circular Buffer Time
10,000	_____	_____
100,000	_____	_____
1,000,000	_____	_____

The actual values must be obtained by executing both programs on the same computer with the same compiler settings.

7.8 F. Defend the Final Design

The circular buffer is selected because it avoids shifting the array whenever a new transaction arrives after the buffer becomes full. It provides constant-time insertion and maintains a fixed memory requirement.

Advantages:

1. Faster insertion for a full transaction history.

2. No need to move all existing transactions.
3. Fixed memory usage.
4. Automatically discards the oldest transaction.
5. Suitable for systems that continuously receive new transactions.

Disadvantages:

1. The implementation is slightly more complex than a simple shifting array.
2. The start index must be managed correctly.
3. Displaying transactions requires calculating positions using modulo.
4. The buffer has a fixed capacity.

The original shifting method may still be preferable when the number of transactions is very small and implementation simplicity is more important than insertion performance.

7.9 Final Conclusion

The redesign challenge demonstrates how an existing C++ program can be improved by analysing its limitations and selecting an appropriate data structure. The original fixed array uses shifting to discard old transactions, which takes O(k) time when full. The redesigned circular buffer performs insertion in O(1) time while maintaining the same fixed storage capacity. Therefore, the circular buffer is a more efficient design for frequently updated transaction histories.

OVERALL CONCLUSION

This assignment demonstrates the practical application of functions and object-oriented programming concepts in C++. It covers call by value, call by reference, call by pointer, return by reference, default arguments, function overloading, classes, arrays inside classes, static members, arrays of objects, friend functions and inline functions.

The programs demonstrate how functions can be designed to modify data safely, how references can avoid unnecessary copying, and how classes can protect data through encapsulation. Static members help maintain information shared by multiple objects, while friend functions support operations that require access to private data from more than one object.

The assignment also demonstrates the importance of analysing time complexity and space complexity. The circular buffer redesign in Q7 shows how a suitable data structure can improve insertion performance without increasing the fixed memory requirement.

Overall, the assignment improves the ability to design, implement, test and evaluate C++ programs using functions and object-oriented programming principles.

REFERENCES

1. Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley.
2. Bjarne Stroustrup, *A Tour of C++*, 3rd Edition, Addison-Wesley.
3. ISO/IEC 14882, *Programming Languages – C++ Standard*.
4. cppreference.com, “Functions in C++.”
5. cppreference.com, “References and Pointers.”
6. cppreference.com, “Classes and Objects.”
7. cppreference.com, “Static Members.”
8. cppreference.com, “Friend Declaration.”

9. GCC Documentation, “Warning Options and Optimisation Options.”
10. Microsoft Learn, “C++ Language Reference.”
